Résolution SUDOKU 2023

Ecole Polytechnique Fédérale de Lausanne

Etudiants	N° SCIPER
Mathélyo FAVRE-REGUILLON	360717
Etienne BOLARD	344705

EPFL

Professeur: Ch. Salzmann
Durée du projet: Septembre, 2023 - Decembre, 2023

Couverture: The Rolex Learning Center at EPFL (Modified)
Style: EPFL Report Style, with modifications by Batuhan Faik Derinbay

1.0 - Description Projet

Résolution d'une grille de Sudoku (9X9) à partir d'une image au format jpeg/jpg prise par l'utilisateur. La solution combine 4 modules orchestrés par LabVIEW:

- Module LabView pour traiter l'image, reconnaître la grille et la découper en 81 cellules.
- Module C pour reconnaître le chiffre d'une cellule.
- Module Matlab pour la résolution de la grille du sudoku.
- Module Labview pour la présentation de la grille résolue.

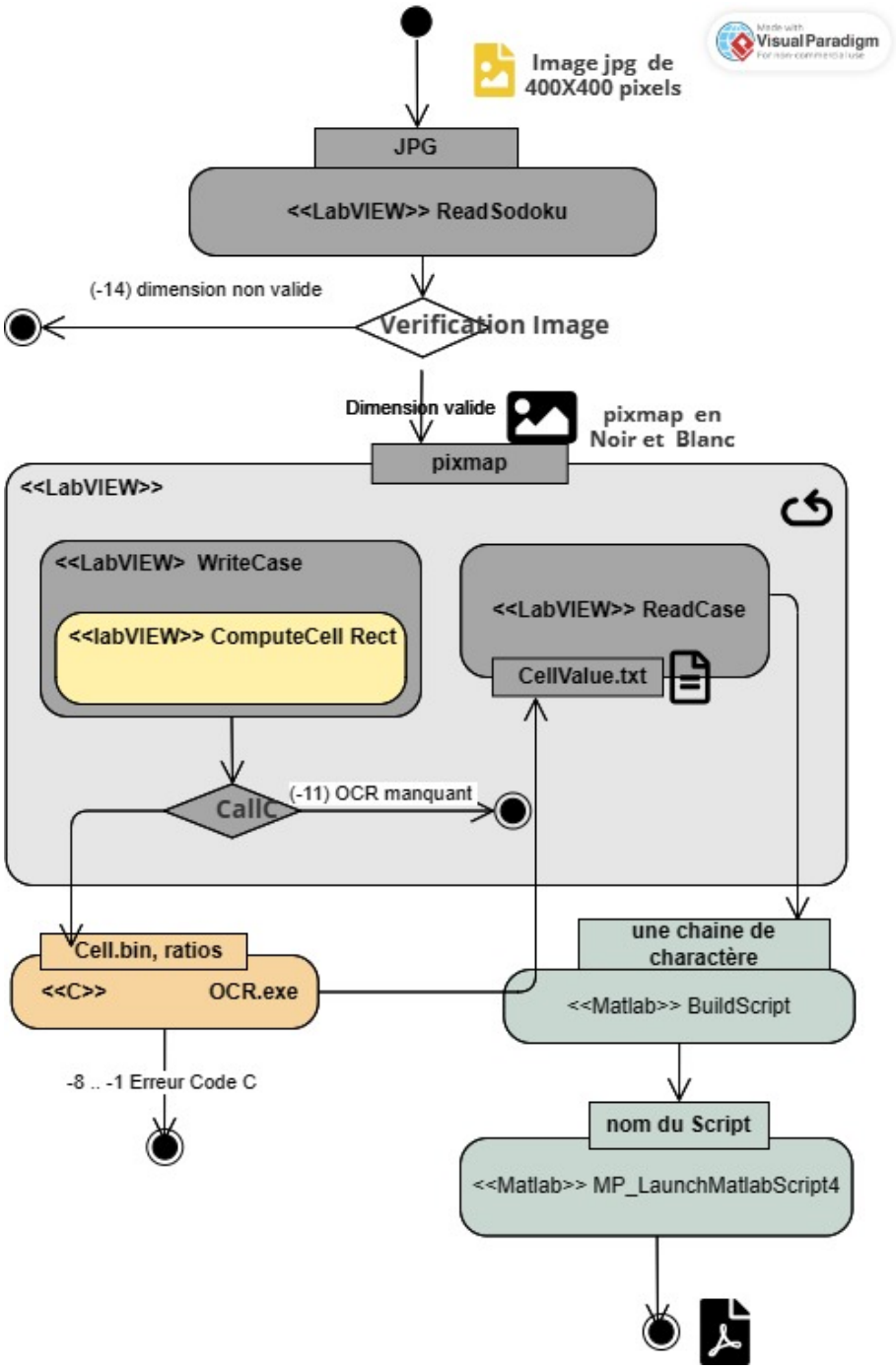

Diagramme de communication des modules

2.0 - Liste des fichiers

Liste des fichiers C utilisés par ce projet:

- **OCR.c:** programme principal de reconnaissance du chiffre dans la cellule

Liste des fichiers LabVIEW utilisés par ce projet:

- **BuildScript.vi:** construit le script Matlab
- **CallC.vi:** appelle OCR pour créer CellValue.txt
- **ComputeCellRect.vi:** donne les pixels limites délimitant une case
- **MP_LaunchMatlabScript4.vi:** exécute le fichier Matlab
- **ReadCase.vi:** crée le pixmap d'une case
- **ReadSudoku.vi:** ouvre le chemin pour afficher l'image et donne les infos de l'image ReadCase
- **SudokuSolver.vi:** prend une image en entrée et sort un pdf du sudoku résolu **(VI principale)**
- **WriteCase.vi:** crée Cell.bin grâce au pixmap dentrée dune case

Liste des fichiers Matlab utilisés par ce projet:

- **checkSolved.m:** vérifie la solvabilité d'une grille de sudoku
- **DispSudoku.m:** crée un pdf du sudoku résolu/irresolvable
- **FillHypothesis.m:** remplit un sudoku avec ses possibilités
- **getChoices.m:** sort les possibilités d'une case
- **solveSudoku.m:** résout de manière itérative le sudoku

Liste des autres fichiers significatifs:

- **OCR.exe:** exécutable
- **CellValue.txt:** fichier contenant la valeur d'une cellule de Cell.bin
- **s1.pdf..s13.pdf:** grilles résolues à la sortie de notre machine
- **Cell.bin:** exemple de cellule

3.0 - Liste des environnements utilisés pour ce projet

- **OS + version:** Windows 10
- **Plateforme de test (VM ou machine perso):** VM
- **Compilateur C + version:** Microsoft Visual Studio 2019
- **Matlab + version:** MATLABR2023
- **LabVIEW + version:** NI LABVIEW 2023 Q3 (64-bits)
- **Documentation C:** générée à partir du code source C avec Doxygen et Sphinx

4.0 - Gestion des codes erreurs

Module	*Code Erreur*	Description de l'erreur
C	*-8 à -1*	Erreur dans le programme C
LabVIEW	*-10*	Document au mauvais format
LabVIEW	*-11*	OCR manquant ou dans le mauvais chemin
LabVIEW	*-12*	Case 'Solve with Matlab' non cochée
LabVIEW	*-13*	Case 'Show pdf' non cochée
LabVIEW	*-14*	Dimensions non valides

5.0 - Références du fichier OCR.c

5.1 - unsigned int GetCellBit (unsigned char * Cellbin, int Width, int line, int col)

Accéde au pixel du chiffre du fichier Cell.bin pour la comparaison. Cette fonction sélectionne et trouve si le pixel donné du chiffre Cell.bin est blanc ou noir. Comme le fichier Cell.bin sort un tableau à une ligne, il faut donc trouver le pixel pour chaque ligne de pixel du chiffre. Comme le fichier à une largeur de Width, il faut à chaque ligne: Width pixels, d'où la multiplication par Width.

Paramètres

in	*Cellbin*	fichier bin contenant le chiffre à déterminer et donc à comparer
in	*Width*	largeur de l'image bin
in	*line*	ligne d'un pixel de l'image bin
in	*col*	colonne d'un pixel de l'image bin
out	*status du pixel*	1 (=noir) ou 0 (=blanc)

5.2 - unsigned int GetDigitBitmapBit (int number, int x, int y)

Accéde au pixel du chiffre modèle pour la comparaison. Cette fonction sélectionne et trouve si le pixel donné d'un chiffre est blanc ou noir en utilisant un décalage bit à bit pour trouver la bonne ligne et la bonne colonne du pixel voulu.

Paramètres

in	*number*	chiffre de référence par rapport au chiffre Cellbin
in	*x*	ligne d'un pixel de l'image de référence
in	*y*	colonne d'un pixel de l'image de référence
out	*status du pixel*	1 (=noir) ou 0 (=blanc)

5.3 - int main (int argc, const char *argv[])

La fonction main contient la création des fichiers et leurs ouvertures; la gestion des erreurs; puis le corps du programme avec la comparaison et la reconnaissance du chiffre du fichier Cell.bin.

Cell.bin est un fichier binaire composé d'une suite de valeurs au format binaire structuré comme ci-dessous: (

Largeur de l'image en pixels
Hauteur de l'image en pixels
pixel 0
pixel 1
...
dernier pixel (Largeur X Hauteur pixels dans le fichier))

Paramètres

in	*argc*	taille du tableau argv[] qui sont les paramètres de la ligne de commande que l'on retrouve dans LABView
in	*argv*	tableau de taille argc, contenant différents éléments, dont le nom du programme, le nom du fichier Cell.bin, et les différents ratios qui sont les pourcentages de seuils des différents chiffres pour être valider
out	*codes*	Valeurs possibles : **0** le chiffre du fichier Cell.bin a été reconnu. Sa valeur est écrite dans le fichier ./CellValue.txt. **-1** si le tableau contient moins de 12 éléments et ferme le programme. Le libellé et le détail de l'erreur sont disponibles sur le canal stderr. **-2** si les ratios du tableau argv[] ne sont pas compris entre l'intervalle 5% et 100% inclus. Le libellé et le détail de l'erreur sont disponibles sur le canal stderr. **-3** si le fichier Cell.bin n'as pas réussi à s'ouvrir. Le libellé et le détail de l'erreur sont disponibles sur le canal stderr. **-4** si le fichier CellValue.txt n'as pas réussi à s'ouvrir ou à se créer. Le libellé et le détail de l'erreur sont disponibles sur le canal stderr. **-5** si l'un des deux premiers paramètres (hauteur et la largeur) du fichier Cell.bin est nul. Le libellé et le détail de l'erreur sont disponibles sur le canal stderr. **-6** si les valeurs de largeur et hauteur de Cell.bin ne sont respectivement pas comprises dans les intervalles ([16, ..., 100] et [24, ..., 100]). Le libellé et le détail de l'erreur sont disponibles sur le canal stderr. **-7** si l'allocation mémoire pour charger Cell.bin ne s'est pas créée correctement. Le libellé et le détail de l'erreur sont disponibles sur le canal stderr. **-8** si matrice de pixel du fichier Cell.bin ne contient pas le résultat du produit largeur par hauteur mentionnés dans les paramètres du fichier. Le libellé et le détail de l'erreur sont disponibles sur le canal stderr.

Etape 1 : case vide
Première vérification : le programme parcourt l'entièreté de la mémoire Cellbin et compare chaque pixel à un pixel blanc, donc un 0. Si c'est le cas, il incrémente count. Une fois tous les pixels comparés, il calculer le ratio de pixel blanc, donc en commun avec la case vide. Si ce ratio est plus grand que le seuil prévu pour la case vide, alors c'est une case vide, et le programme s'arrête.

Etape 2 : comparaison aux autres chiffres 1 à 9
Si le programme ne reconnaît pas directement la case vide, il exécute la boucle suivante pour chaque chiffre. Il compare alors pixel par pixel selon le fichier DigitBitmap. Mais comme le fichier DigitBitmap contient des cellules ayant un format 16x24, il faut donc appliquer des offsets pour la ligne et la colonne offset et effectuer la comparaison. Après chaque configuration 16x24 possible, le nombre de pixel en commun est stocké dans idem_max si ce dernier est plus petit que idem. Après avoir réalisée la comparaison pour chaque comparaison d'un chiffre, il l'exécute pour un autre chiffre. Après chaque chiffre, il calcule le ratio de pixel en commun avec la configuration la plus optimale avec la valeur de idem_max. Il faut également que ce ratio soit plus grand que le seuil de son chiffre, mais qu'il soit également le plus grand si plusieurs chiffres sont plus grand que leur seuil. Le chiffre est alors stocké dans chiffre_reconnu.

Si aucun des chiffres n'a satisfait les conditions dites précédemment, alors chiffre_reconnu vaut toujours 0, et alors on a une case vide.

6.0 - Références des fichiers LabVIEW.vi

6.1 - ReadSudoku.vi

ReadSudoku permet d'ouvrir un chemin menant à un document au format .jpg. Il va trouver les spécifications de l'image comme sa hauteur et sa largeur en pixel. Si la hauteur ou la largeur de l'image ne vaut pas exactement 400 pixels, ReadSudoku envoie l'erreur 'dimensions non valides' avec le code '-14'. Si cette condition est respectée, ReadSudoku sort l'image en noir et blanc et des informations sur un pixmap créé a partir de cette image.

6.2 - WriteCase.vi

WriteCase prend en entrée les données d'un pixmap et celles d'un cluster représentant une case de sudoku en pixel. Ce VI va isoler la case dans le pixmap et créer un fichier de booléans nommé 'Cell.bin' représentant les pixels vides de cette image par 'False' et les pixels pleins par 'True'.

6.3 - ComputeCellRect.vi

ComputeCellRect prend en entrée les variables des boucles 'for' représentant dans le cas d'un sudoku ses lignes et ses colonnes. Il sort un cluster de données qui correspondent aux pixels délimitant la case de sudoku située grace à ses lignes et ses colonnes.

6.4 - CallC.vi

CallC vérifie que l'executable 'OCR.exe' est présent dans le directoire courant. Si celui-ci est présent, ce Vi utilise le terminal de commande pour lancer l'exécutable qui va permettre de reconnaître le chiffre de la case du sudoku précedemment isolée. 'OCR.exe' crée un fichier nommé 'TextValue' au format '.txt' avec le chiffre reconnu et son pourcentage de reconnaissance.

6.5 - ReadCase.vi

ReadCase va isoler dans le fichier 'CellValue.txt' le chiffre déterminé précedemment par l'executable 'OCR.exe' et va le concaténer dans un string. Ce string est vide a la première itération des boucles 'for' et est rempli par les chiffres identifié suivi d'une virgule.

6.6 - BuildScript.vi

BuildScript prend en entrée la chaine de charactère créée précedemment par ReadCase et le chemin du fichier image. Ce VI crée une chaine de charactère de commande Matlab en transformant la chaine de charactère en entrée en un tableau et en isolant le nom du fichier en entrée.

6.7 - MP_LaunchMatlabScript4.vi

Launch matlab with the script passed as parameter. The script and the VI MUST be in the same directory! Works on OSX, Windows and Linux On Linux, assume Ubuntu (salles CO) and current 2013 "matlab" CMD. On OSX, search for the latest release of matlab, see diagram. On Windows, assume "matlab" command exists. There is demo script if needed, see the diagram to activate it. ChS. 2013-21, updated to use '-batch' instead of '-r'